



PARCIAL 2

Integrantes:

Isabela Cabezas Obregon – 2226013

Isabella Ortiz Hernández - 2226215

Samuel Sepúlveda Castaño - 2232026

Presentado a:

Oscar Mondagron

**Universidad Autónoma de Occidente
Facultad de Ingeniería y Ciencias básicas
Servicios Telematicos
Santiago de Cali, Colombia
Abril de 2026**

Configure UFW en el servidor Ubuntu como punto obligatorio de control de tráfico. Establezca la política por defecto en deny para todo el tráfico entrante. En ningún caso se permitirá el acceso directo al servicio FTPS sin pasar por el firewall.

```
vagrant@servidor:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
```

```
vagrant@servidor:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
```

Habilite únicamente los puertos estrictamente necesarios para el funcionamiento de FTPS: puerto 21 (canal de control), rango de puertos pasivos 50000:50010 (canal de datos) y puerto 22 (administración SSH). No deben quedar puertos adicionales abiertos

```
vagrant@servidor:~$ sudo ufw allow 22/tcp
Skipping adding existing rule
Skipping adding existing rule (v6)
vagrant@servidor:~$ sudo ufw allow 21/tcp
Rule updated
Rule updated (v6)
vagrant@servidor:~$ sudo ufw allow 50000:50010/tcp
Rule added
Rule added (v6)
vagrant@servidor:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
```

Demuestre el control de acceso del firewall: con UFW activo, pero sin la regla del puerto 21 configurada, intente conectarse desde el cliente al servidor (la conexión debe ser rechazada). A continuación, agregue la regla correspondiente y confirme que el acceso es exitoso.

```
vagrant@servidor:~$ sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
21/tcp ALLOW Anywhere
50000:50010/tcp ALLOW Anywhere
22/tcp (v6) ALLOW Anywhere (v6)
21/tcp (v6) ALLOW Anywhere (v6)
50000:50010/tcp (v6) ALLOW Anywhere (v6)
```

```
vagrant@servidor:~$ sudo ufw delete allow 21/tcp
Rule deleted
Rule deleted (v6)
```

```
vagrant@cliente:~$ ftp 192.168.50.3
ftp: Can't connect to '192.168.50.3:21': Connection refused
ftp: Can't connect to '192.168.50.3:ftp'
ftp> |
```

```

Rule added (v6)
vagrant@servidor:~$ sudo ufw allow 21/tcp
Rule added
Rule added (v6)

```

```

vagrant@cliente:~$ ftp 192.168.50.3
Connected to 192.168.50.3.
220 (vsFTPd 3.0.5)
Name (192.168.50.3:vagrant):

```

Muestre el estado completo de UFW con `sudo ufw status verbose` y explique el propósito de cada regla activa en el contexto del servicio FTPS.

```

vagrant@servidor:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), allow (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
50000:50010/tcp ALLOW IN Anywhere
21/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
50000:50010/tcp (v6) ALLOW IN Anywhere (v6)
21/tcp (v6) ALLOW IN Anywhere (v6)

```

22/tcp	acceso remoto (SSH / SFTP)
21/tcp	canal de control FTP
50000–50010	canal de datos en modo pasivo

Instale y configure `vsftpd` para operar en modo FTPS con TLS explícito. Reutilice el certificado de servidor y la clave privada generados en clase con OpenSSL.

```

vagrant@servidor:~$ cat /etc/vsftpd.conf | grep ssl
ssl_enable=YES
allow_anon_ssl=NO
force_local_data_ssl=YES
force_local_logins_ssl=YES
rsa_cert_file=/etc/ssl/vsftpd/vsftpd.crt
rsa_private_key_file=/etc/ssl/vsftpd/vsftpd.key

vagrant@servidor:~$ cat /etc/vsftpd.conf | grep rsa
rsa_cert_file=/etc/ssl/vsftpd/vsftpd.crt
rsa_private_key_file=/etc/ssl/vsftpd/vsftpd.key

```

Configure el rango de puertos pasivos en `vsftpd` (`pasv_min_port=50000`, `pasv_max_port=50010`) de forma consistente con las reglas UFW definidas. Explique la relación entre ambas configuraciones y por qué deben coincidir para que la transferencia de archivos funcione.

```
vagrant@servidor:~$ cat /etc/vsftpd.conf | grep pasv
pasv_enable=YES
pasv_min_port=50000
pasv_max_port=50010
```

```
vagrant@servidor:~$ sudo ufw status
Status: active
```

To	Action	From
--	-----	----
22/tcp	ALLOW	Anywhere
50000:50010/tcp	ALLOW	Anywhere
21/tcp	ALLOW	Anywhere
22/tcp (v6)	ALLOW	Anywhere (v6)
50000:50010/tcp (v6)	ALLOW	Anywhere (v6)
21/tcp (v6)	ALLOW	Anywhere (v6)

En modo pasivo, el servidor FTP abre conexiones de datos en un rango específico de puertos. Si estos puertos no están permitidos en el firewall (UFW), el cliente no podrá establecer la conexión de datos, lo que impide operaciones como listar, subir o descargar archivos.

Desde un cliente FTP con soporte TLS (FileZilla u equivalente) en modo TLS Explícito, conéctese al servidor FTPS. Importe el certificado de la CA para que el cliente valide correctamente el certificado del servidor. Realice las siguientes operaciones y documente cada una: listar el directorio remoto, subir un archivo desde el cliente al servidor y descargarlo de vuelta.

```
vagrant@cliente:~$ lftp -u isabela,1105 ftp://192.168.50.3
lftp isabela@192.168.50.3:~> set ftp:ssl-force true
lftp isabela@192.168.50.3:~> set ftp:ssl-protect-data true
lftp isabela@192.168.50.3:~> set ssl:verify-certificate no
lftp isabela@192.168.50.3:~> ls
lftp isabela@192.168.50.3:~> ls
lftp isabela@192.168.50.3:~> !echo "hola parcial" > prueba.txt
lftp isabela@192.168.50.3:~> put prueba.txt
13 bytes transferred
lftp isabela@192.168.50.3:~> ls
-rw----- 1 1004 1004 13 Apr 03 21:40 prueba.txt
lftp isabela@192.168.50.3:~>
```

```
lftp isabela@192.168.50.3:~> get prueba.txt
get: /home/vagrant/prueba.txt: file already exists and xfer:clobber is unset
lftp isabela@192.168.50.3:~> !ls
comprimido.html index.html.1 index.html.3 index.html.5 index.html.7 index.html.9 normal.html
index.html      index.html.2 index.html.4 index.html.6 index.html.8 main.html      prueba.txt
```

Verifique la negociación TLS desde la línea de comandos ejecutando: `openssl s_client -connect [IP_SERVIDOR]:21 -starttls ftp`. Confirme que el certificado presentado corresponde al servidor configurado y que la cadena de confianza es válida (Verify return code: 0).

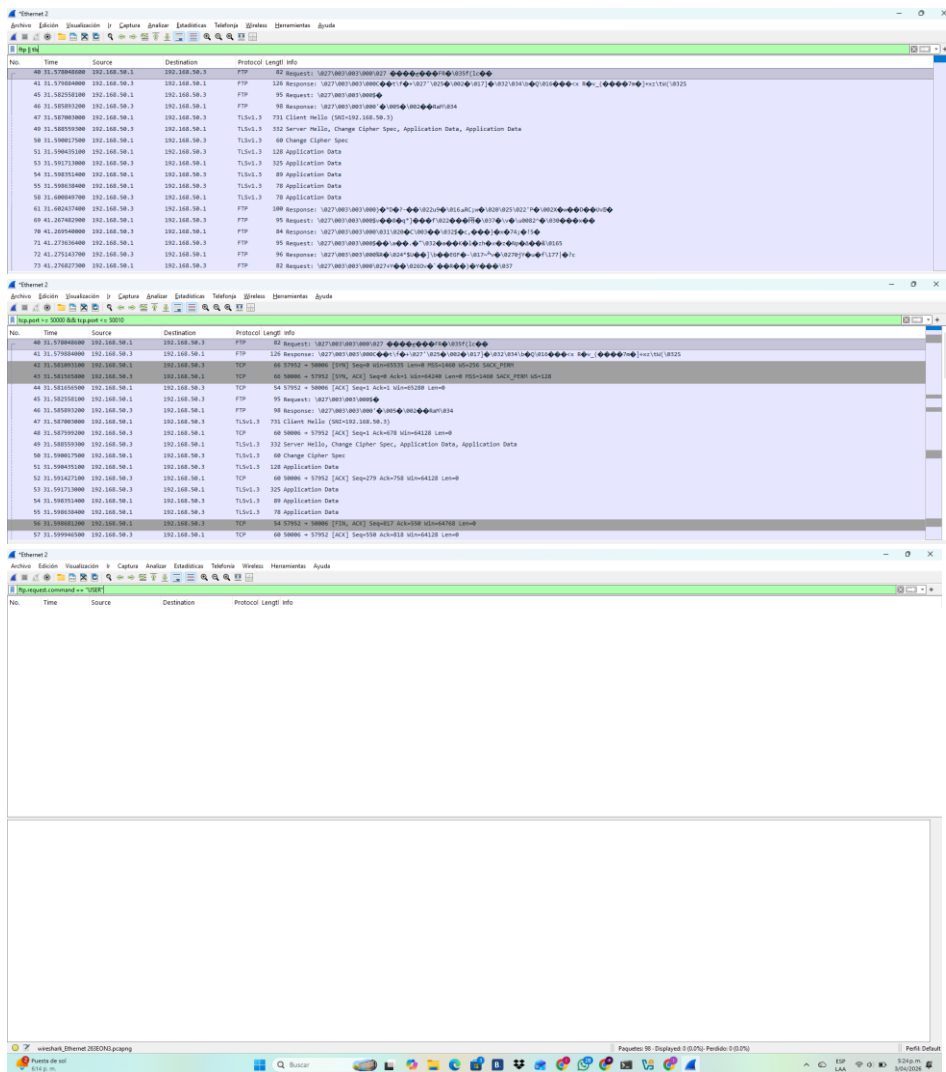
```
subject=C = CO, ST = Valle del Cauca, L = Cali, O = UAO, OU = Ingenieria, CN = 192.168.50.3
issuer=C = CO, ST = Valle del Cauca, L = Cali, O = UAO, OU = Ingenieria, CN = 192.168.50.3
```

```
SSL-Session:
Protocol : TLSv1.3
```

```
Start Time: 1775254544
Timeout : 7200 (sec)
Verify return code: 0 (ok)
Extended master secret: no
Max Early Data: 0

====
read R BLOCK
```

Capture con Wireshark el tráfico generado durante una sesión FTPS activa. Identifique el handshake TLS en el canal de control (puerto 21) y el tráfico cifrado en el canal de datos (puertos pasivos). Demuestre que las credenciales y el contenido transferido son ilegibles en la captura.



SEGUNDA PARTE

Configure la maquina cliente Linux para utilizar DNS sobre TLS (DoT) editando el archivo `/etc/systemd/resolved.conf`. Establezca los campos `DNS=`, `FallbackDNS=` y `DNSOverTLS=yes` con al menos dos servidores DNS públicos con soporte DoT (por ejemplo 1.1.1.1 y 8.8.8.8). Reinicie el servicio con `sudo systemctl restart systemd-resolved`.

```
# Quad9: 9.9.9.9#DNS:quad9.net 149.112.112.112#DNS
DNS=1.1.1.1 8.8.8.8
FallbackDNS=1.0.0.1 8.8.4.4
#Domains=
#DNSSEC=no
DNSOverTLS=yes
```

Verifique el estado de la configuración ejecutando `resolvectl status`. Confirme que el campo DNS over TLS aparece como yes y que los servidores configurados son los correctos. Muestre la salida completa del comando.

```
vagrant@cliente:~$ resolvectl status
Global
  Protocols: -LLMNR -mDNS +DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: foreign
  DNS Servers: 1.1.1.1 8.8.8.8
  Fallback DNS Servers: 1.0.0.1 8.8.4.4

Link 2 (eth0)
Current Scopes: DNS
  Protocols: +DefaultRoute +LLMNR -mDNS +DNSOverTLS DNSSEC=no/unsupported
  DNS Servers: 1.1.1.1 8.8.8.8 192.168.50.3 200.21.200.10 200.21.200.80
  DNS Domain: ~.

Link 3 (eth1)
Current Scopes: none
  Protocols: -DefaultRoute +LLMNR -mDNS +DNSOverTLS DNSSEC=no/unsupported
```

Realice consultas DNS de prueba con `resolvectl query [dominio]` o `dig` y confirme que la resolución es exitosa para al menos tres dominios distintos.

```
vagrant@cliente:~$ resolvectl query google.com
google.com: 2800:3f0:4005:40a::200e -- link: eth0
            172.217.162.110 -- link: eth0

-- Information acquired via protocol DNS in 66.8ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: yes
-- Data from: network
vagrant@cliente:~$ resolvectl query facebook.com
facebook.com: 2a03:2880:f102:8a:face:b00c:0:25de -- link: eth0
              57.144.114.1 -- link: eth0

-- Information acquired via protocol DNS in 144.9ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: yes
-- Data from: network
vagrant@cliente:~$ resolvectl query ubuntu.com
ubuntu.com: 2620:2d:4000:1::28 -- link: eth0
            2620:2d:4000:1::26 -- link: eth0
            2620:2d:4000:1::27 -- link: eth0
            185.125.190.21 -- link: eth0
            185.125.190.20 -- link: eth0
            185.125.190.29 -- link: eth0

-- Information acquired via protocol DNS in 32.1ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: yes
-- Data from: network
```

Capture el tráfico de red con Wireshark durante una consulta DNS con DoT activo. Aplique el filtro de puerto 853 y confirme que las consultas viajan cifradas sobre TLS. Capture también una consulta DNS convencional sobre el puerto 53 (puede deshabilitar temporalmente DoT para esto). Muestre ambas capturas y explique qué información queda expuesta en el tráfico DNS sin cifrar.

DNS sobre TLS (puerto 853) (DNSOverTLS=yes):

El tráfico se encuentra cifrado mediante TLS, por lo que no es posible visualizar el contenido de las consultas ni las respuestas, únicamente metadatos como IP origen/destino y tamaño de paquetes.

The screenshot displays a Wireshark packet capture of DNS traffic over TLS on port 853. The packet list shows several packets, with the selected packet (No. 391) being a TLSv1.3 packet. The packet details pane shows the TLS structure, including the Client Hello and Server Hello messages. The packet bytes pane shows the raw data, which is encrypted. The terminal window on the right shows the output of the 'sudo resolvectl flush-caches' and 'resolvectl query openai.com' commands, indicating that the DNS query was successful and the response was received via protocol DNS in 162.5ms.

DNS sin cifrar (puerto 53) (DNSOverTLS=no):

Las consultas y respuestas DNS viajan en texto claro, permitiendo observar dominios consultados, direcciones IP y tipo de registros.

The screenshot displays a Wireshark packet capture of DNS traffic on port 53. The packet list shows several packets, with the selected packet (No. 148) being a Standard query packet. The packet details pane shows the DNS structure, including the query and response messages. The packet bytes pane shows the raw data, which is in plaintext. The terminal window on the right shows the output of the 'sudo nano /etc/systemd/resolved.conf', 'sudo systemctl restart systemd-resolved', and 'resolvectl query openai.com' commands, indicating that the DNS query was successful and the response was received via protocol DNS in 278.8ms.

TERCERA PARTE

Configure el servidor OpenSSH para permitir conexiones SFTP.

```
vagrant@servidor:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-04-03 23:19:24 UTC; 14s ago
     Docs: man:sshd(8)
           man:sshd_config(5)
    Main PID: 3472 (sshd)
      Tasks: 1 (limit: 3425)
     Memory: 1.7M
        CPU: 37ms
    CGroup: /system.slice/ssh.service
            └─3472 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Apr 03 23:19:24 servidor systemd[1]: Starting OpenBSD Secure Shell server...
Apr 03 23:19:24 servidor sshd[3472]: Server listening on 0.0.0.0 port 22.
Apr 03 23:19:24 servidor sshd[3472]: Server listening on :: port 22.
Apr 03 23:19:24 servidor systemd[1]: Started OpenBSD Secure Shell server.
vagrant@servidor:~$ cat /etc/ssh/sshd_config | grep sftp
Subsystem       sftp            /usr/lib/openssh/sftp-server
```

Ajuste las reglas UFW para el servicio SFTP: verifique que el puerto 22 este habilitado (configurado desde la Primera Parte). Demuestre que intentar una transferencia SFTP con el puerto 22 bloqueado en UFW falla, y que al habilitarlo la transferencia es exitosa.

```
vagrant@servidor:~$ sudo ufw deny 22
Rule added
Rule added (v6)
```

```
vagrant@servidor:~$ sudo ufw status numbered
Status: active

    To      Action    From
    --      -
[ 1] 50000:50010/tcp    ALLOW IN  Anywhere
[ 2] 21/tcp            ALLOW IN  Anywhere
[ 3] 22                DENY IN   Anywhere
[ 4] 22/tcp (v6)        ALLOW IN  Anywhere (v6)
[ 5] 21/tcp (v6)        ALLOW IN  Anywhere (v6)
[ 6] 22 (v6)           DENY IN   Anywhere (v6)
```

```
vagrant@cliente:~$ sftp isabela@192.168.50.3
ssh: connect to host 192.168.50.3 port 22: Connection refused
Connection closed.
Connection closed
```

```
vagrant@servidor:~$ sudo ufw allow 22
Rule updated
Rule updated (v6)
```

```
vagrant@cliente:~$ sftp isabela@192.168.50.3
isabela@192.168.50.3's password:
Connected to 192.168.50.3.
sftp> |
```

Desde el cliente Linux, conéctese al servidor mediante SFTP usando el comando `sftp [usuario]@[IP_SERVIDOR]`. Realice las siguientes operaciones y documente cada una

con capturas: listar el directorio remoto con ls, subir un archivo con put y descargarlo con get.

```
vagrant@cliente:~$ sftp isabela@192.168.50.3
isabela@192.168.50.3's password:
Connected to 192.168.50.3.
sftp> ls
parcial2.txt  prueba.txt
```

```
vagrant@cliente:~$ touch prueba2.txt
vagrant@cliente:~$ sftp isabela@192.168.50.3
isabela@192.168.50.3's password:
Connected to 192.168.50.3.
sftp> put prueba2.txt
Uploading prueba2.txt to /home/isabela/prueba2.txt
prueba2.txt                                100%   0   0.0KB/s   00:00
sftp> |
```

```
sftp> get prueba2.txt
Fetching /home/isabela/prueba2.txt to prueba2.txt
```

Capture con Wireshark el tráfico generado durante una sesión SFTP activa aplicando el filtro tcp.port == 22. Demuestre que a diferencia de FTP, todo el tráfico de SFTP (autenticación y datos) viaja cifrado dentro de un único canal SSH. Compare visualmente con la captura de FTP sin cifrar obtenida en la Primera Parte.

Ethernet 2

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

tcp.port == 22

No.	Time	Source	Destination	Protocol	Length	Info
42	6.281985280	192.168.50.3	192.168.50.1	SSHv2	108	Server: Encrypted packet (len=52)
43	6.284090400	192.168.50.1	192.168.50.3	SSHv2	108	Client: Encrypted packet (len=52)
44	6.286122400	192.168.50.1	192.168.50.3	SSHv2	1108	Server: Encrypted packet (len=1052)
45	6.287796200	192.168.50.1	192.168.50.3	SSHv2	108	Client: Encrypted packet (len=52)
46	6.289900400	192.168.50.3	192.168.50.1	SSHv2	122	Server: Encrypted packet (len=68)
47	6.211953200	192.168.50.1	192.168.50.3	SSHv2	108	Client: Encrypted packet (len=52)
48	6.211693900	192.168.50.3	192.168.50.1	SSHv2	122	Server: Encrypted packet (len=68)
49	6.211658000	192.168.50.1	192.168.50.3	TCP	54	62185 → 22 [ACK] Seq=2451 Ack=4233 Win=5280 Len=0
135	24.060776400	192.168.50.1	192.168.50.3	SSHv2	138	Client: Encrypted packet (len=84)
136	24.062924000	192.168.50.3	192.168.50.1	SSHv2	108	Server: Encrypted packet (len=52)
137	24.065372000	192.168.50.1	192.168.50.3	SSHv2	108	Client: Encrypted packet (len=52)
138	24.066701500	192.168.50.3	192.168.50.1	SSHv2	122	Server: Encrypted packet (len=68)
139	24.070979700	192.168.50.1	192.168.50.3	TCP	54	62185 → 22 [ACK] Seq=2587 Ack=4352 Win=5924 Len=0
221	36.481622200	192.168.50.1	192.168.50.3	SSHv2	138	Client: Encrypted packet (len=76)
222	36.483675700	192.168.50.3	192.168.50.1	SSHv2	138	Server: Encrypted packet (len=76)
223	36.486333900	192.168.50.1	192.168.50.3	SSHv2	138	Client: Encrypted packet (len=76)
224	36.487709300	192.168.50.3	192.168.50.1	SSHv2	138	Server: Encrypted packet (len=76)
225	36.489460800	192.168.50.1	192.168.50.3	SSHv2	138	Client: Encrypted packet (len=84)
226	36.490847000	192.168.50.3	192.168.50.1	SSHv2	108	Server: Encrypted packet (len=52)
227	36.492343100	192.168.50.1	192.168.50.3	SSHv2	122	Client: Encrypted packet (len=68)
228	36.494316300	192.168.50.1	192.168.50.3	SSHv2	122	Server: Encrypted packet (len=68)
229	36.495861500	192.168.50.1	192.168.50.3	SSHv2	108	Client: Encrypted packet (len=52)
230	36.497204000	192.168.50.3	192.168.50.1	SSHv2	122	Server: Encrypted packet (len=68)
231	36.537640900	192.168.50.1	192.168.50.3	TCP	54	62185 → 22 [ACK] Seq=2943 Ack=4691 Win=64768 Len=0

> Frame 1: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on Interface vDeviceVNF_3329908A-34EC-4B1D-9776-B921B

> Ethernet II, Src: 0a:00:27:00:00:05 (0a:00:27:00:00:05), Dst: PCSsystemec_e8:06:1a (08:00:27:0e:06:1a)

> Internet Protocol Version 4, Src: 192.168.50.1, Dst: 192.168.50.3

> Transmission Control Protocol, Src Port: 62185, Dst Port: 22, Seq: 8, Len: 0

vagrant@servidor: ~

vagrant@cliente: ~

sftp> get prueba2.txt
Fetching /home/isabela/prueba2.txt to prueba2.txt
sftp> exit
vagrant@cliente:~\$ sftp isabela@192.168.50.3
isabela@192.168.50.3's password:
Connected to 192.168.50.3.
sftp> ls
parcial2.txt prueba.txt prueba2.txt
sftp> put prueba2.txt
Uploading prueba2.txt to /home/isabela/prueba2.txt
prueba2.txt 100% 0 0.0
KB/s 00:00
sftp> get prueba2.txt
Fetching /home/isabela/prueba2.txt to prueba2.txt
sftp> exit
vagrant@cliente:~\$ sftp isabela@192.168.50.3
isabela@192.168.50.3's password:
Connected to 192.168.50.3.
sftp> ls
parcial2.txt prueba.txt prueba2.txt
sftp> exit
vagrant@cliente:~\$ sftp isabela@192.168.50.3
isabela@192.168.50.3's password:
Connection closed by 192.168.50.3 port 22
Connection closed.
vagrant@cliente:~\$ sftp isabela@192.168.50.3
isabela@192.168.50.3's password:
Connected to 192.168.50.3.
sftp> ls
parcial2.txt prueba.txt prueba2.txt
sftp> put prueba2.txt
Uploading prueba2.txt to /home/isabela/prueba2.txt
prueba2.txt 100% 0 0.0
KB/s 00:00
sftp> get prueba2.txt
Fetching /home/isabela/prueba2.txt to prueba2.txt
sftp>

SFTP	FTP
------	-----

utiliza SSH (puerto 22) y cifra toda la comunicación en un único canal. No es posible observar credenciales ni contenido de archivos en la captura.	las credenciales y datos viajan en texto claro, SFTP protege toda la comunicación mediante cifrado.
---	---

Elabore una tabla comparativa entre FTPS y SFTP con los siguientes criterios: protocolo base, numero de puertos utilizados, tipo de certificado requerido, modo de cifrado, compatibilidad con firewalls y facilidad de configuración. Concluya cuál de los dos considera más adecuado para un entorno con restricciones de firewall estrictas y por qué.

CRITERIOS	FTPS	SFTP
Protocolo Base	FTP + TLS	SSH
Numero de Puertos Utilizados	21 + múltiples puertos dinámicos	22
Certificado Requerido	SSL/TLS	No requiere
Modo de Cifrado	Parcial (control + datos)	Total
Compatibilidad Firewalls	Alta ya que requiere configurar TLS, certificados y puertos pasivos	Baja porque solo configurar SSH
Facilidad de Configuración	Más compleja, propensa a errores	Más simple y directa

SFTP es más adecuado para entornos con restricciones estrictas de firewall, ya que utiliza un único puerto (22), lo que simplifica la configuración y reduce la superficie de ataque. Además, encapsula toda la comunicación dentro de un canal SSH cifrado, garantizando la confidencialidad tanto de las credenciales como de los datos transferidos. En contraste, FTPS requiere múltiples puertos y una configuración más compleja, lo que puede generar problemas en redes con políticas de seguridad restrictivas.